패킷 네트워크 상호 통신을 위한 프로토콜 (TCP/IP의 기원)

> 원제: *A Protocol for Packet Network Intercommunication*

항목	내용
저자	Vinton G. Cerf (Stanford University), Robert E. Kahn (DARPA, 미국)
학술지	*IEEE Transactions on Communications*, Vol. 22, No. 5, pp. 637-648, 1974
분야	컴퓨터 네트워크 · 인터넷 구조
난이도	★★☆☆☆
찾는 법	DOI: 10.1109/TCOM.1974.1092259 (IEEE Xplore, 대학 도서관 경유)
PDF 원문	PDF (보스턴대 강의 자료) · 공식본은 IEEE Xplore
ai_life 연결 단원	04 AI 에이전트(API·통신 구조)

한 줄 요약

서로 다른 패킷 네트워크들을 '게이트웨이'로 연결하고, 종단 호스트끼리 신뢰성 있게 통신하는 공통 프로토콜(TCP)을 제안 — 오늘날 인터넷 구조의 설계도.

왜 중요한가

두 저자는 이 논문으로 2004년 튜링상을 받았습니다. AI 공부와도 연결됩니다: 거대 모델의 **분산 학습**, 클라우드 추론, 연합학습(federated learning)은 모두 이 논문이 정한 "종단 간(end-to-end)" 원칙 위에서 작동합니다.

핵심 내용

1. 문제: 이질적인 네트워크

각 네트워크마다 패킷 크기, 속도, 오류율, 주소 체계가 다름. 이를 **하나로 통일하지 않고** 연결하려면?

2. 핵심 설계 원칙

원칙	설명
게이트웨이	네트워크 사이에서 패킷을 전달(필요시 분할)만 함 — 내부는 단순하게
종단 간 신뢰성	오류 복구·순서 보장은 네트워크가 아닌 **양 끝 호스트의 TCP**가 책임
계층적 주소	네트워크 번호 + 호스트 번호 → 오늘날 IP 주소의 원형
분할(fragmentation)	작은 패킷만 허용하는 네트워크를 지날 때 쪼개고 목적지에서 재조립

3. 신뢰성 메커니즘

- **순서 번호(sequence number)**: 바이트 단위로 번호를 붙여 순서 복원·중복 제거
- **확인 응답(ACK)과 재전송**: 일정 시간 내 ACK가 없으면 다시 보냄
- **슬라이딩 윈도우 흐름 제어**: 수신자가 받을 수 있는 양(윈도우)만큼만 전송

> 💡 수학적 관점: 슬라이딩 윈도우의 처리량은 대략 $처리량 \approx \frac{윈도우\ 크기}{왕복\ 시간(\mathrm{RTT})}$ — 간단한 모델이지만 네트워크 성능 분석의 기본.

4. 연결 설정과 해제

양쪽이 초기 순서 번호를 맞추는 절차를 논의 → 이후 **3-way handshake**로 정착.

읽는 순서 가이드

1. **서론 + Figure 1~2** - 네트워크·게이트웨이 구조 (15분)
2. **게이트웨이 / 분할 절** - 패킷 형식 그림 위주 (20분)
3. **TCP 절 (순서 번호, 재전송, 윈도우)** - 가장 중요 (40분)
4. **연결 설정** - 현대 TCP와 비교하며 (15분)

사전 지식 체크

- [ ] 패킷 교환 vs 회선 교환
- [ ] OSI / TCP-IP 계층 구조 (현대 교과서 기준)
- [ ] 큐잉(대기행렬) 기초 — 선택

생각해볼 질문

1. 1974년 논문에는 TCP와 IP가 하나였다. 왜 나중에 둘로 분리되었을까?
2. "네트워크는 단순하게, 지능은 끝에" 라는 원칙은 오늘날 AI 엣지 컴퓨팅 / 온디바이스 AI와 어떻게 닮았는가?
3. 패킷 손실이 있을 때 재전송 타이머를 어떻게 정해야 할까? 이것을 통계적 추정 문제로 바꿔 보자. (힌트: Jacobson 1988, RTT 지수이동평균 — Adam과 같은 아이디어!)

용어 정리

영어	한국어
gateway	게이트웨이 (현재의 라우터)
fragmentation / reassembly	분할 / 재조립
sequence number	순서 번호
acknowledgment (ACK)	확인 응답
sliding window	슬라이딩 윈도우
end-to-end principle	종단 간 원칙

다음에 이어 읽을 논문

- Saltzer, Reed, Clark (1984), *End-to-End Arguments in System Design*
- Jacobson (1988), *Congestion Avoidance and Control*

✍️ 나의 메모